



MERRIMACK COLLEGE

CSC 6033

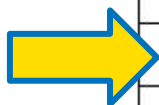
Week 7

Processing Input and Output

Languages, Automata, and Decidability - Dr. Paulo Fernandes

Timeline

Where are we?



Unit	Week	Topic	Projects	Quizzes	In-class Exercises	Exam
1	1	From Python to C++	P#1	Q#1	E#1	
1	2	Developing with C++	P#2	Q#2	E#2	
2	3	Finite Automata	P#3	Q#3	E#3	
2	4	Regular Expressions	P#4	Q#4	E#4	
3	5	Pushdown Automata	P#5	Q#5	E#5	
4	6	Turing Machines	P#6	Q#6	E#6	
5	7	Processing Input and Output	P#7	Q#7	E#7	
6	8	Decidability				Final Exam (all units)



MERRIMACK COLLEGE

Deadlines are hard now in the end of the course.

Presentation Agenda

Week 7

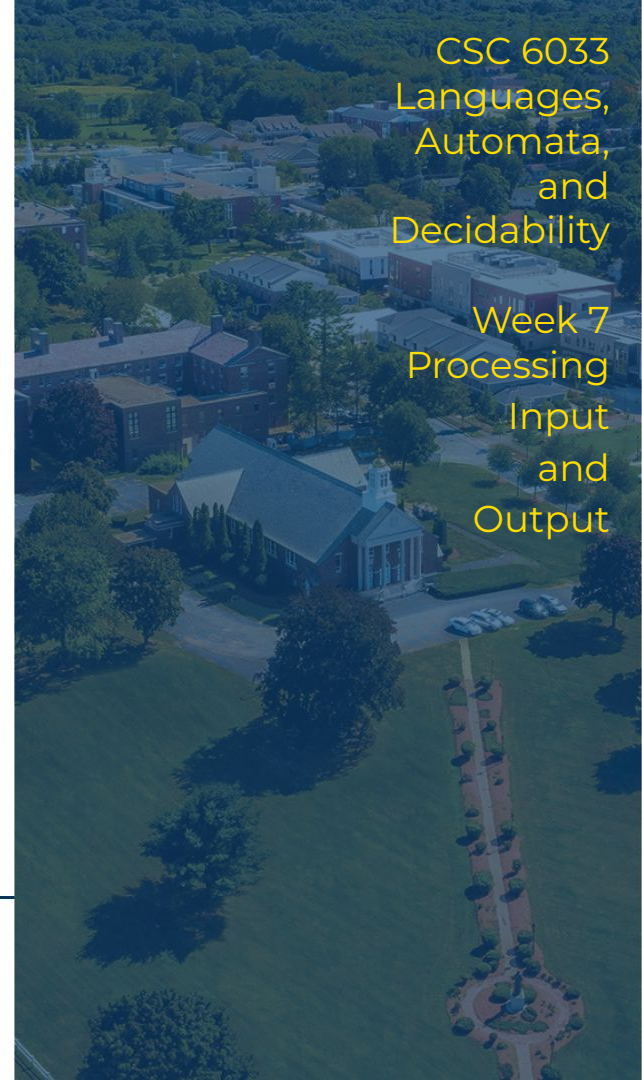
- Turing Machines with Input and Output
 - a. Beyond Acceptance
 - b. Kinds of Output
- Turing Machines Examples
 - a. Translation Examples
 - b. Computation Examples
- This Week's tasks



MERRIMACK COLLEGE

CSC 6033
Languages,
Automata,
and
Decidability

Week 7
Processing
Input
and
Output



TM's with Input and Output

“For every complex problem there is an answer that is clear, simple, and wrong.”

H. L. Mencken

FSA and PDA were basically used to decide if a word belongs to a language.

TM can be used this way too, but since the tape may hold information, TM can have more than just a boolean answer.

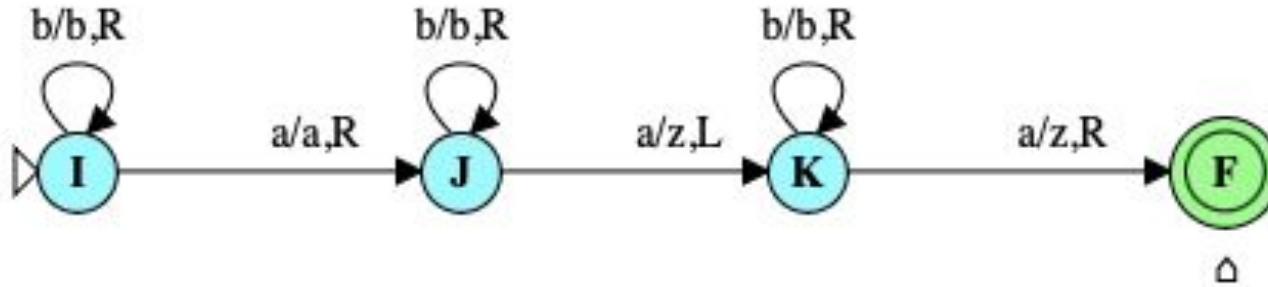
In such way, TM is not only a machine to recognize if a word is acceptable, but to perform processing with a richer output.

- **Beyond Acceptance**
- Kinds of Output



Turing Machines - TM

For example, the TM below accepts words where the first letter **a** has to be followed by another letter **a**.



b	a	a	b	a
---	---	---	---	---



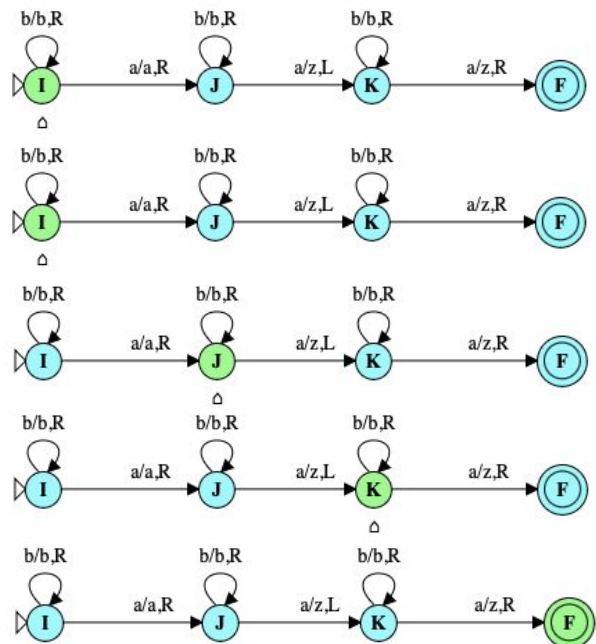
MERRIMACK COLLEGE

Tool: [Formal Language Editor - Old Dominion Univ.](#)

Turing Machines - TM



b	a	a	b	a
---	---	---	---	---



b	a	a	b	a
---	---	---	---	---

b	a	a	b	a
---	---	---	---	---

b	a	a	b	a
---	---	---	---	---

b	a	z	b	a
---	---	---	---	---

b	z	z	b	a
---	---	---	---	---



The initial word in the tape, **baabb**, is accepted because the first letter **a** is followed by another letter **a**.

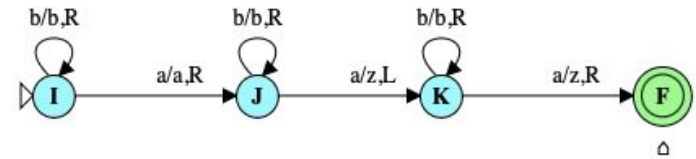
At the end the tape shows the first two letter **a**'s become **z**, and the head points to the next letter.



MERRIMACK COLLEGE

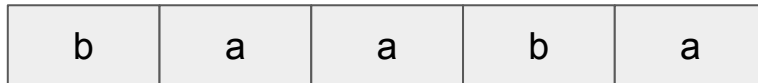
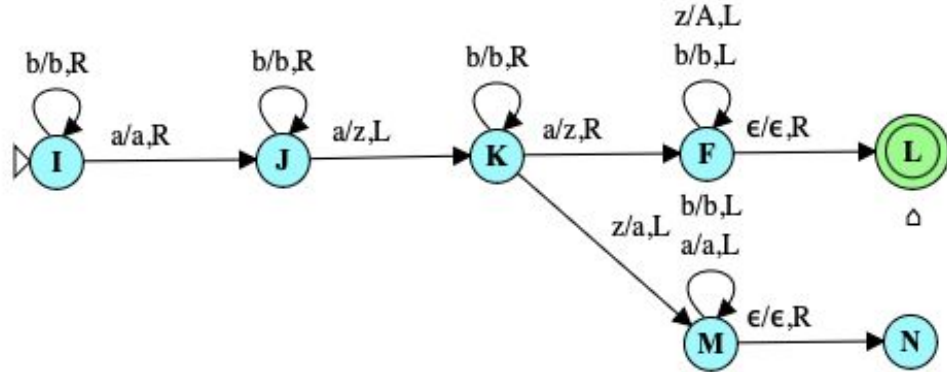
Execution towards acceptance.

TM - Beyond acceptance



For example, the TM below accepts words where the first letter **a** has to be followed by another letter **a**.

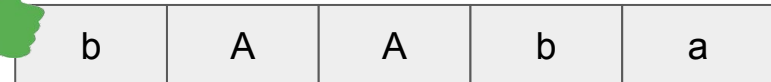
This new TM verifies the acceptance, but also indicates with capital **A**'s the pair of input **a**'s used to verify acceptance.



input



output



MERRIMACK COLLEGE

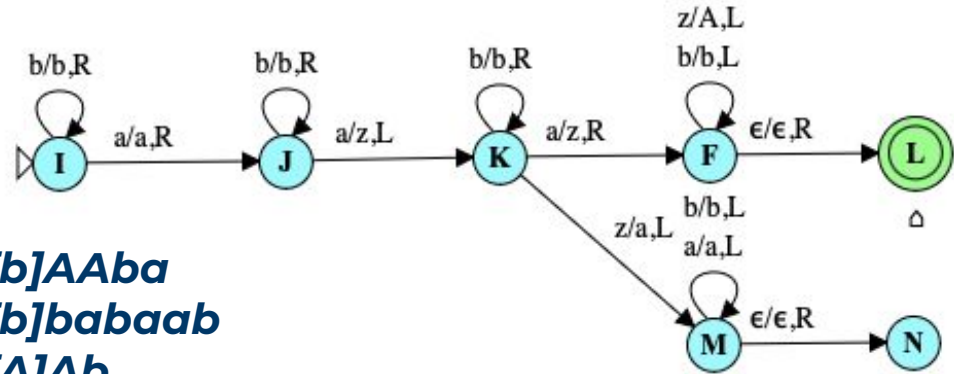
Execution caring about the output.

TM - Beyond acceptance

Execute this TM for the words as input:

- **[b]aaba** - accepted - **[b]AAba**
- **[b]babaab** - rejected - **[b]babaab**
- **[a]ab** - accepted - **[A]Ab**
- **[a]bba** - rejected - **[a]bba**
- **[b]baabbaa** - accepted - **[b]bAAbbaa**

You should observe the result in terms of acceptance or rejection, and the resulting content of the tape.



TM's with Input and Output

“For every complex problem there is an answer that is clear, simple, and wrong.”

H. L. Mencken

FSA and PDA were basically used to decide if a word belong to a language.

TM can be used this way too, but since the tape may hold information, TM can have more than just a boolean answer.

In such way, TM is not only a machine to recognize if a word is acceptable, but to perform processing with a richer output.

- Beyond Acceptance
- **Kinds of Output**



Kinds of TM output

All TM have the traditional output of:

- acceptance, if a final state is reached; or
- rejection, otherwise.

- Acceptance TM:
 - To see if a word does or does not belong to a language.
- Translation TM:
 - To find and replace patterns.
- Computation TM:
 - To perform an operation.

We may also want to pay attention to other forms of output through the content of the tape, or which tape cell is under the head after the final move.

In general we have two tape-based outputs:

- translation behavior, where basically a specific input is translated into a specific output;
- computation behavior, where basically some transformation to the input is reflected in the output.



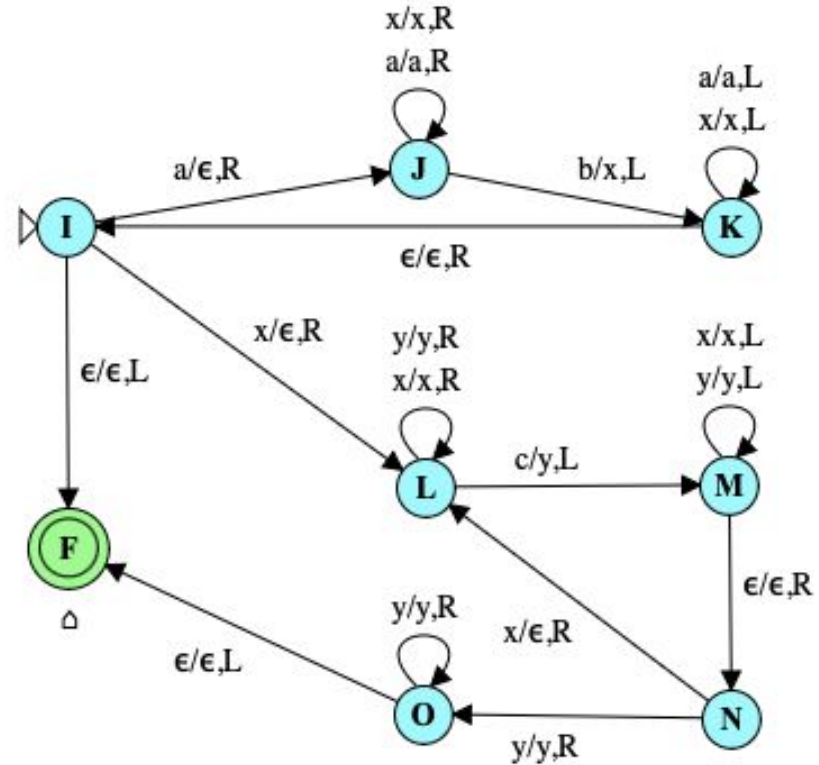
Acceptance TM

If a TM **only** cares about reaching the final state (ignoring the content of the tape and position of the head), it is considered an **Acceptance TM**.

For example, this TM is used to verify if an input word belongs to $a^n b^n c^n$ language $a^n b^n c^n$ where $n \geq 0$.

Result:

- **[a]abbcc** - accepted - **y[y]**
- **[a]bbcc** - rejected - **[b]cc**
- **[a]abbccc** - rejected - **yy[c]**



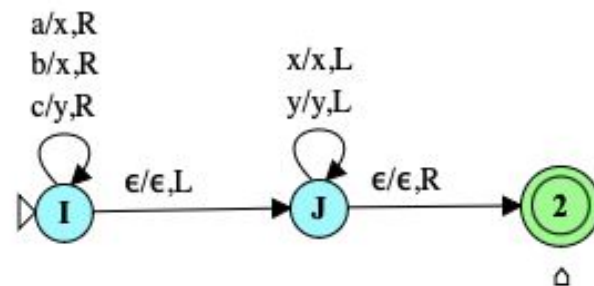
Translation TM

If a TM cares about reaching the final state, and, if the input word is accepted, delivers a translation of the input in the remaining content of the tape, such a TM is called a **Translation TM**.

For example, this TM is used to accept words of the language $(a/b/c)^*$ and for those words it delivers a word that replaces all letters **a** and **b** with the letter **x**, and the letters **c** with the letter **y**.

Result:

- **[a]abbcc** - accepted - **[x]xxxxyy**
- **[b]acad** - rejected - **xxyx[d]**
- **[c]cbacac** - accepted - **[y]yxxxyxy**



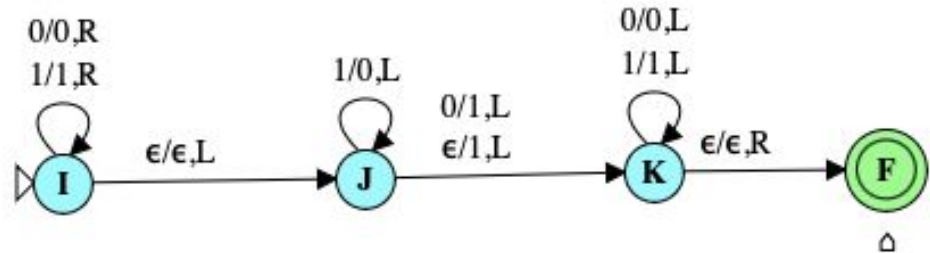
Computation TM

If a TM cares about reaching the final state, and, if the input word is accepted, delivers an operation (often, but not necessarily, arithmetic) that results in the remaining content of the tape, such a TM is called a **Computation TM**.

For example, this TM is used to accept words of the language $(0/1)^*$ encoding a binary number, and for those words it delivers a binary number that is one more than the input.

Result:

- **$[1]01$** - accepted - **$[1]10$**
- **$[1]11$** - accepted - **$[1]000$**
- **$[1]100$** - accepted - **$[1]101$**



TM's Examples

“A computer is a machine for constructing mappings from input to output.”

Michael Kirby

The use of Turing Machines with input and output is a powerful tool. In fact, it's as powerful as any programming language.

This does not mean that it is as easy to code in a Turing Machine as in a high-level programming language, but it means that everything that can be done with a programming language, can be done by a Turing Machine, and vice-versa.

- **Translation Examples**
- Computation Examples

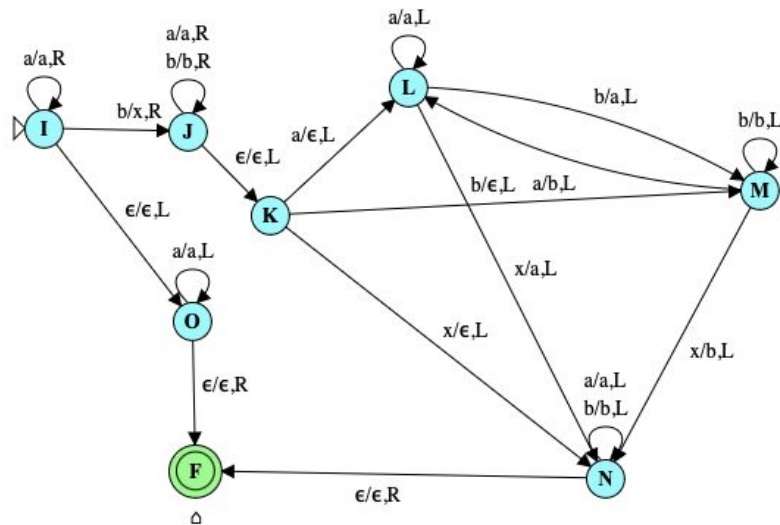


TM - Translation Example

Try this TM for the input **aabbbaa** [here](#), then try some more.

The TM here accepts words with **a**'s and **b**'s, and we will remove the first letter **b** found.

- It starts searching for the first **b** (State **I**);
- When found, we replace it with the symbol **x** (State **J**);
- It goes to the end of the word and starts copying symbols from right to left until removing the symbol **x** (States **K**, **L**, and **M**);
- It returns to the beginning of the word (State **N**) - State **O** deals with no letter **b**.



TM's Examples

“A computer is a machine for constructing mappings from input to output.”

Michael Kirby

The use of Turing Machines with input and output is a powerful tool. In fact, it's as powerful as any programming language.

This does not mean that it is as easy to code in a Turing Machine as in a high-level programming language, but it means that everything that can be done with a programming language, can be done by a Turing Machine, and vice-versa.

- Translation Examples
- **Computation Examples**



TM - Computation Example

The next TM accepts words with two binary numbers (**0**'s and **1**'s) separated by a letter **a**.

- The execution performs a decrement of the second number (OP2) and an increment of the first number (OP1) until the second number becomes zero.
- For example, if we want to add 13 and 5 the tape starts with:
 - **1101a101**
 - The TM would decrement OP2 and increment OP1 five times until erasing OP2 and the addition symbol (a).
 - The resulting tape should display:
 - **10010**

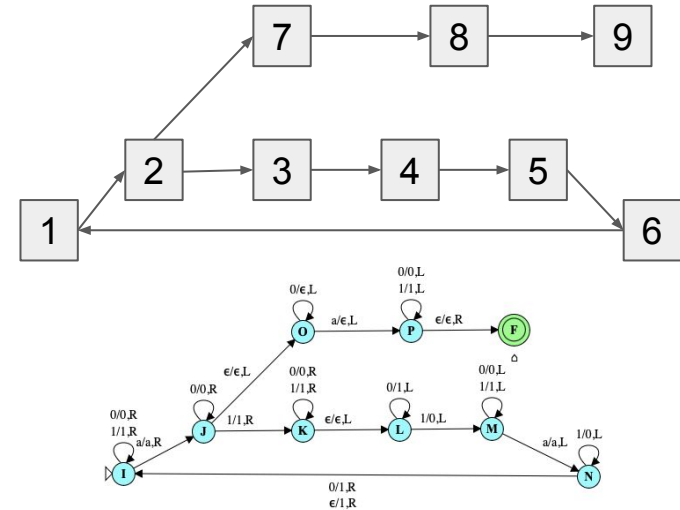


TM - Computation Example

input: **1101a101**
output: **10010**

The execution performs a decrement of the second number (OP2) and an increment of the first number (OP1) until the second number becomes zero. The TM performs:

1. Go to the beginning of OP2;
2. If OP2 is zero go to Step 7;
3. If OP2 is not zero go to the end of OP2;
4. Subtract one from OP2;
5. Go to the end of OP1;
6. Add one to OP1 and return to Step 1;
7. Erase OP2 and addition symbol **a**;
8. Return the head to the beginning of OP1;
9. Stop the execution.



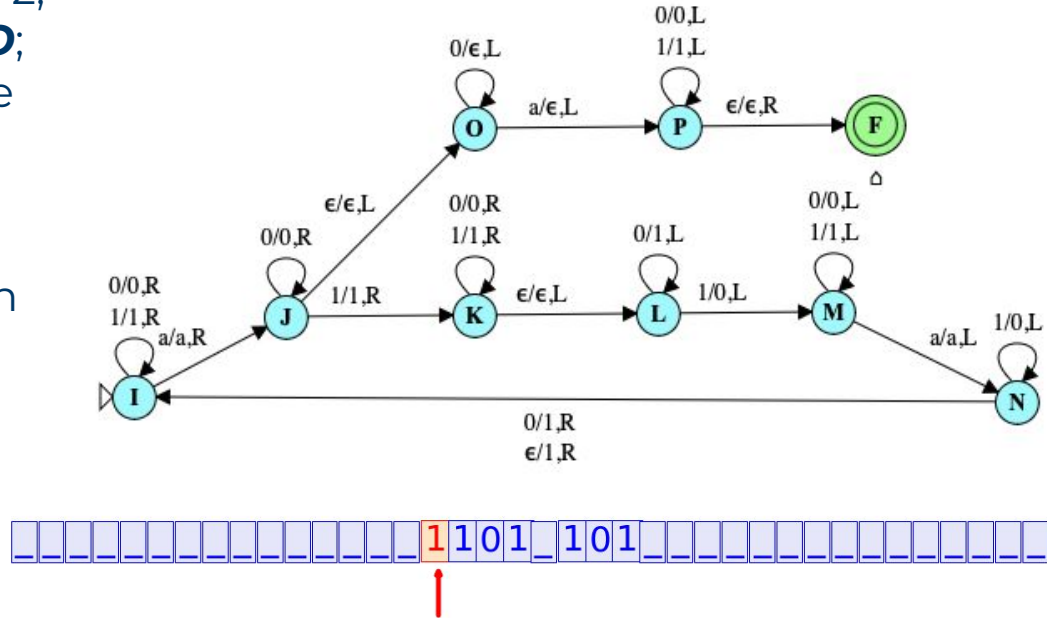
MERRIMACK COLLEGE

The steps described are represented by the same number of states in a TM.

TM - Computation Example

- I. Go to the beginning of OP2;
- J. If OP2 is zero go to State **O**;
- K. If OP2 is not zero go to the end of OP2;
- L. Subtract one from OP2;
- M. Go to the end of OP1;
- N. Add one to OP1 and return to State **I**;
- O. Erase OP2 and addition symbol **a**;
- P. Return the head to the beginning of OP1;
- F. Stop the execution.

input: **1101a101**
output: **10010**



MERRIMACK COLLEGE

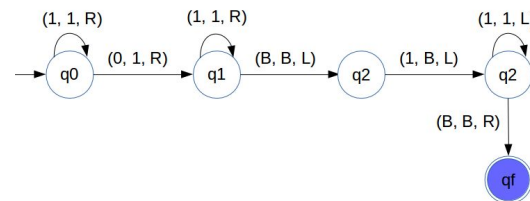
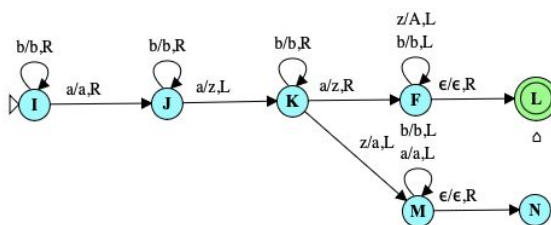
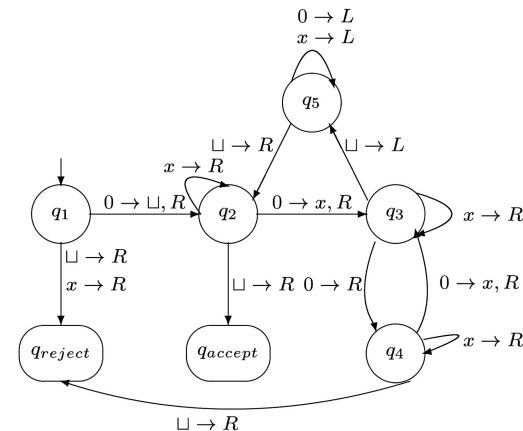
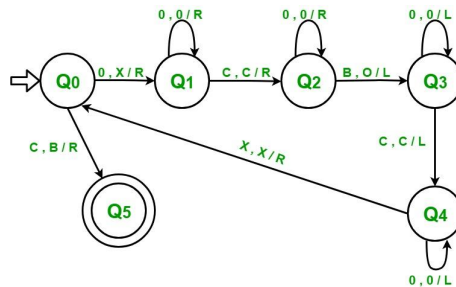
Try this TM for the input example [here](#)
(then, try other numbers).

TM - Computation Example

Examples of turing machines are abundant in the literature (and also the Internet).

However, the notation is not uniform.

While searching for additional examples, pay attention to the specific notation employed.



This Week's tasks

- In-class Exercise E#7
- Coding Project P#7
- Quiz Q#7

Tasks

- Execute the step-by-step of execution of the TM described in slides 17 to 19 for the binary numbers 1101 and 11.
- Create a Turing Machine that receives two unary numbers and add it.
- Quiz #7 about this week topics.



In-class Exercise - E#7

Considering the TM example of binary sum (slides 17 to 19) do the step-by-step of execution for the binary numbers 1101 and 11.

- Feel free to use the Formal Language Editor Tool to execute it;
- Write down the current state of the tape (including the head position) and indicate the current state of the TM at each step;
- You can write it down by yourself or take screenshots of your execution.

Your step-by-step execution should be depicted in a single **.pdf** file.

You have to submit the **.pdf** file with your work on Canvas by the deadline.



MERRIMACK COLLEGE

This task counts towards the In-class Exercises grade
and the deadline is This Saturday.

Seventh Coding Project - P#7

Write a C++ program that executes the operation as stated by the TM for addition of two binary numbers (slides 17 to 19).

- Your code should receive two binary numbers and output the resulting sum (also in binary).
- Make sure your code mimics the TM operations, dealing with the binary numbers as a string of characters 1 and 0, and following the logic to increase the first number and decreasing the second one.

Try your TM for the following examples: 1101 and 101, resulting in 10010; and 1101 and 11, resulting in 10000.

You have to submit the **.cpp** file with your work on Canvas by the deadline.



MERRIMACK COLLEGE

This assignment counts towards the Projects grade and the deadline is Next Tuesday.

Seventh Quiz - Q#7

- The seventh quiz in this course covers the topics of Week 7;
- The quiz will be available this Saturday, and it is composed of 10 questions;
- The quiz should be taken on Canvas (Module 7), and it is not timed:
 - You can take as long as you want to answer it;
- The quiz is open book, open notes, and you can even use any language interpreter to answer it;
- Yet, the quiz is evaluated and you are allowed to submit it only once.

Your task:

- Go to Canvas, answer the quiz and submit it by the deadline.



MERRIMACK COLLEGE

This quiz counts towards the Quizzes grade and the deadline is Next Tuesday.

” **Week 7 of CSC 6033**

- **Do In-class Exercise E#7 by Saturday;**
- **Do Quiz Q#7 (available Saturday) by next Tuesday;**
- **Develop Coding Project P#7 by next Tuesday.**

Next Week content - Decidability and the final exam



MERRIMACK COLLEGE